

Universidade de Vila Velha – UVV

Curso de Ciência da Computação

Disciplina: Desenvolvimento Web

Sala: CC5MB

HelpNOW

Plataforma Web de Serviços Domésticos

Documentação Técnica do Projeto

Integrantes:

Antony Novais

Heitor Rodrigues

Thomás Kriger

Professor:

Wanderson Muniz

Vila Velha, junho de 2026

Conteúdo

1	Descrição Geral do Produto	2
1.1	Problema Endereçado	2
1.2	Funcionalidades Principais	2
2	Tecnologias Utilizadas	3
2.1	Justificativa da Stack	3
3	Arquitetura Adotada	3
3.1	Application Factory	3
3.2	Estrutura de Pastas	4
4	Diagrama do Banco de Dados	6
4.1	Modelo Entidade-Relacionamento	6
4.2	Decisão de Design: Estrutura N:M entre Usuário e Papel	8
5	Detalhamento Técnico das Principais Partes do Código	8
5.1	Modelos SQLAlchemy – Typed Mapped Columns	8
5.2	Blueprints e Modularização de Rotas	10
5.2.1	Blueprint de Autenticação	10
5.3	Formulários e Validação com Flask-WTF	11
5.4	Consulta Agregada – Avaliação Média	11
5.5	Frontend – Templates Jinja2 com Bulma	12
5.6	Configuração por Ambiente e Variáveis de Ambiente	13
6	Testes Automatizados	14
6.1	Estratégia de Testes	14
7	Qualidade de Código e Boas Práticas	15
8	Como Rodar o Projeto (Via Github)	16
8.1	Pré-requisitos	16
8.2	Passo a Passo	16
9	Principais Desafios e Como Foram Resolvidos	17
9.1	Validação Condicional por Papel no Formulário de Cadastro	17
9.2	Número de Endereço como String	17
9.3	Desativação do CSRF nos Testes	17
10	Conclusão	18

1 Descrição Geral do Produto

O **HelpNOW** é uma plataforma web desenvolvida em Flask que conecta *clientes* que precisam de serviços domésticos a *prestadores* autônomos cadastrados. O sistema elimina a dependência de indicações informais, centralizando em uma interface simples o ciclo completo de contratação: anúncio do serviço, busca, solicitação e avaliação.

1.1 Problema Endereçado

No cotidiano de hoje, encontrar profissionais confiáveis para serviços como elétrica, hidráulica, pintura, limpeza e informática ainda depende, em grande parte, de redes de contato pessoais. O HelpNOW propõe uma solução digital acessível que organiza essa demanda e amplia o alcance dos profissionais autônomos.

1.2 Funcionalidades Principais

- Cadastro com dois perfis distintos: **Cliente** e **Prestador**, com validações e regras específicas por papel;
- Prestadores anunciam serviços com categoria, descrição, preço e localidade;
- Clientes buscam profissionais filtrando por **categoria** e/ou **localidade**;
- Fluxo completo de solicitação: envio de mensagem e endereço pelo cliente; aceite ou recusa pelo prestador;
- Clientes marcam serviços como **concluídos** e atribuem uma **nota de 1 a 5**;
- Painel do prestador exibe média real de avaliações calculada via consulta SQL agregada;
- Edição de perfil para ambos os tipos de usuário.

2 Tecnologias Utilizadas

Tabela 1: Stack tecnológico do HelpNOW

Camada	Tecnologia
Backend	Python 3.11, Flask, Flask-Login, Flask-WTF
ORM / Banco	SQLAlchemy 2.x, Flask-SQLAlchemy, SQLite
Frontend	Jinja2, Bulma CSS 0.9.4, Font Awesome 6
Formulários	WTForms, Flask-WTF (com CSRF)
Testes	pytest, pytest-flask, pytest-cov
Automação	Invoke (<code>tasks.py</code>)
Configuração	python-dotenv, arquivos <code>.env.*</code>

2.1 Justificativa da Stack

O projeto final do **Helpnow** herda diretamente da proposta e da última versão compartilhada que havia sido estabelecida e desenvolvida em conjunto durante as aulas do bimestre em cima de um app de delivery. A escolha pelo **Flask** se alinha diretamente com o conteúdo ministrado na disciplina, permitindo demonstrar todos os padrões exigidos na avaliação (Application Factory, Blueprints, Flask-WTF, Flask-Login, SQLAlchemy). O **Bulma** foi escolhido por ser um CSS framework moderno, responsivo, sem dependência de JavaScript e com boa semântica HTML. O banco **SQLite** foi adotado para desenvolvimento e testes por ser zero-configuração; a variável `DATABASE_URL` permite trocar para PostgreSQL em produção sem alteração de código.

3 Arquitetura Adotada

3.1 Application Factory

O projeto implementa o padrão *Application Factory* do Flask. A função `create_app()` centraliza toda a inicialização da aplicação, recebendo um dicionário opcional de configurações para sobrescrever valores em tempo de teste. Essa prática é essencial para evitar o famoso problema de importação circular, além de ser o padrão recomendado de arquitetura pela documentação **oficial** do Flask.

```

1 import logging
2 from flask import Flask
3
4 def create_app(test_config=None):
5     app = Flask(__name__)
6 
```

```

7     if app.debug:
8         app.logger.setLevel(logging.DEBUG)
9
10    from helpnow.ext.cli import init_app as init_cli
11    from helpnow.ext.config import init_app as init_config
12    init_cli(app)
13    init_config(app)
14
15    if test_config:
16        app.config.update(test_config)
17
18    from helpnow.ext.db import init_app as init_db, register_models
19    init_db(app)
20    register_models()
21
22    if not app.config.get("TESTING"):
23        from helpnow.ext.wtf import init_app as init_wtf
24        init_wtf(app)
25
26    from helpnow.ext.login import init_app as init_login
27    from helpnow.ext.debugtoolbar import init_app as init_toolbar
28    from helpnow.views import init_app as init_site
29    init_login(app)
30    init_toolbar(app)
31    init_site(app)
32
33    return app

```

Listing 1: app.py – Application Factory

3.2 Estrutura de Pastas

```

helpnow/
|-- app.py                # Application Factory (create_app)
|-- pyproject.toml        # Dependencias e metadados
|-- tasks.py              # Automacao com Invoke
|-- .env.dev / .env.test / .env.prod
|
|-- helpnow/
|   |-- ext/              # Extensoes Flask
|   |   |-- config/       # Carregamento de variaveis de ambiente
|   |   |-- db/           # Inicializacao do SQLAlchemy

```

```

|   |   |-- login/           # Flask-Login (user_loader)
|   |   |-- wtf/            # CSRF via Flask-WTF
|   |   |-- cli/            # Comandos personalizados (flask
create-db)
|   |   `-- debugtoolbar/   # Debug Toolbar (somente dev)
|   |
|   |-- models/             # Modelos SQLAlchemy
|   |   |-- user.py         # User (UserMixin, hashing de senha)
|   |   |-- role.py         # Role (papel: Cliente / Prestador)
|   |   |-- role_user.py    # Tabela associativa User <-> Role
|   |   |-- location.py     # Address e City
|   |   |-- servico.py      # Servico (anuncio do prestador)
|   |   `-- solicitacao.py  # Solicitacao (pedido do cliente)
|   |
|   |-- views/              # Blueprints e rotas
|   |   |-- main.py         # Rotas principais (index, buscar,
servicos)
|   |   |-- auth.py         # Autenticacao (login, cadastro, logout)
|   |   `-- perfil.py       # Edicao de perfil
|   |
|   `-- forms/              # Formularios WTForms
|       |-- auth.py         # LoginForm, CadastroForm
|       |-- servico.py      # ServicoForm, SolicitacaoForm
|       |-- perfil.py       # PerfilForm
|       `-- main.py         # ContatoForm
|
|-- templates/main/         # Templates Jinja2
|-- static/css/             # CSS personalizado (helpnow.css)
`-- tests/                  # Testes automatizados

```

Listing 2: Estrutura de diretórios do projeto

4 Diagrama do Banco de Dados

4.1 Modelo Entidade-Relacionamento

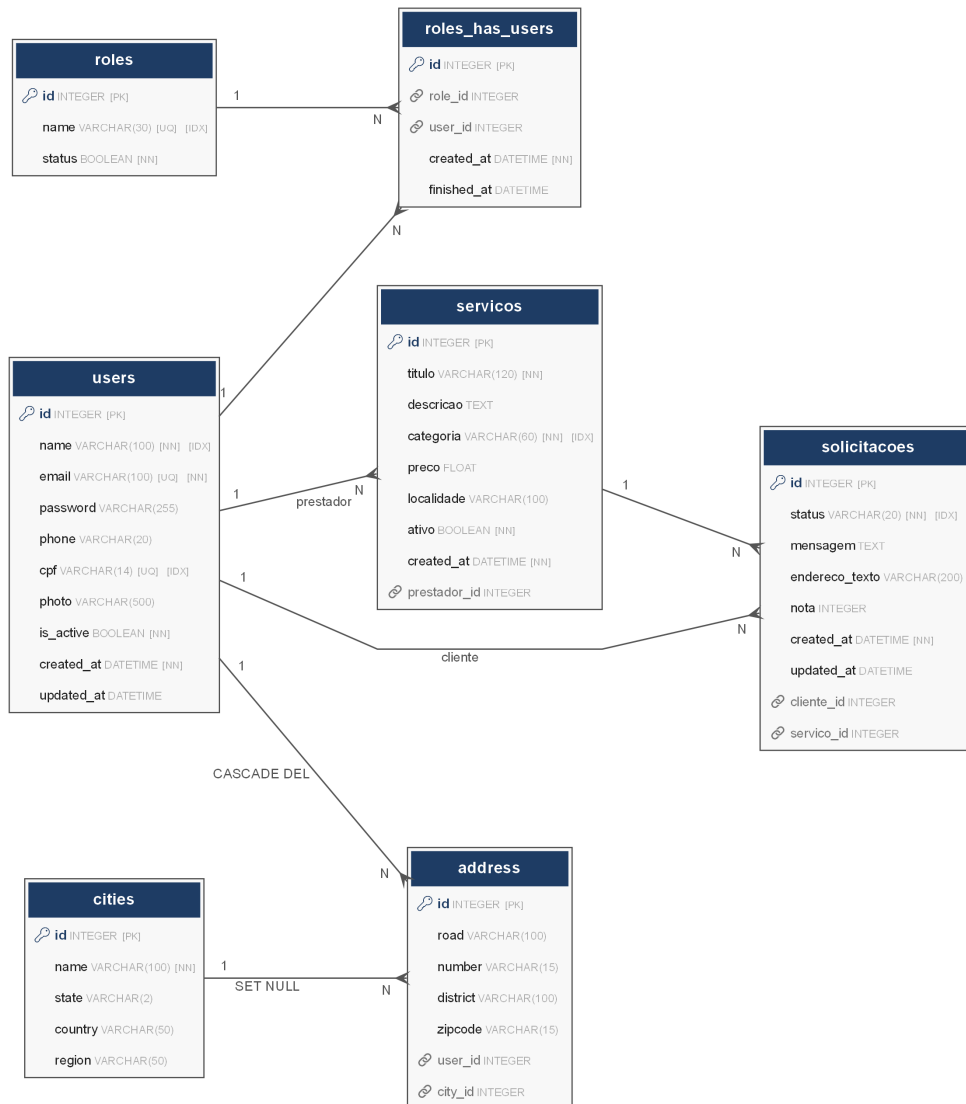


Figura 1: Diagrama ER do HelpNOW

id	name	email	password	phone	cpf	photo	is_active	created_at
1	Thomas Kriger	thomas@email.com	crypt32768b:162WGCIA03lavuz543169d540394a4da...	(27) 99999-1111	444.444.444-44	https://i.postimg.cc/MG594v8/WhatsApp-Image-2026...	✓	2026-06-06 07:...
2	Heitor Eletricista	heitor@email.com	crypt32768b:15nqus7H2NkQ5ACF8\$eebb010d3f1f8e...	(27) 99999-2222	555.555.555-55	https://media.gettyimages.com/id/2061516149/pf/foto/...	✓	2026-06-06 07:...
3	Antony Informática	antony@email.com	crypt32768b:158W8tpaifVidobhsy1b344386c5269...	(27) 99999-3333	666.666.666-66	https://cdn.sistemaweb.com.br/arquivos/cdb118ca80ea...	✓	2026-06-06 07:...

Figura 2: Tabelas geradas pelo SQLAlchemy

Entidades e relações principais

- **users** — dados do usuário (nome, e-mail, senha hash, CPF, foto, telefone, datas de auditoria);
- **roles** — papéis disponíveis: *Cliente* e *Prestador*;
- **roles_has_users** — tabela associativa com restrição de unicidade (*user_id*, *role_id*) e campo *finished_at* para histórico de papéis;
- **cities** — cidades com estado e país (índice composto *name* + *state*);
- **address** — endereços vinculados a um usuário e a uma cidade (CASCADE delete);
- **servicos** — anúncios dos prestadores (título, categoria, preço, localidade, flag *ativo*);
- **solicitacoes** — pedidos dos clientes (status, mensagem, endereço de atendimento, nota de avaliação).

A tabela 2 resume as relações entre entidades.

Tabela 2: Relações entre entidades

De	Tipo	Para	Observação
users	N:M	roles	via roles_has_users
users	1:N	address	CASCADE delete
users	1:N	servicos	FK prestador_id
users	1:N	solicitacoes	FK cliente_id
servicos	1:N	solicitacoes	FK servico_id
cities	1:N	address	SET NULL on delete

4.2 Decisão de Design: Estrutura N:M entre Usuário e Papel

Um detalhe importante que vale ressaltar é que, como percebemos, a tabela `roles_has_users` implementa um relacionamento N:M entre `users` e `roles`. É relevante notar que, na versão atual do sistema, a regra de negócio aplicada é exclusiva: no momento do cadastro, o usuário escolhe ser **Cliente** ou **Prestador**, e essa escolha não pode ser alterada pela interface.

Ainda assim, a estrutura N:M foi mantida intencionalmente por dois motivos:

- **Extensibilidade:** a arquitetura permite que, em versões futuras, um usuário migre de papel sem perda de histórico. O campo `finished_at` na tabela associativa foi projetado exatamente para isso: ao trocar de papel, o registro atual recebe uma data de encerramento e um novo é criado, preservando o histórico completo;
- **Demonstração de modelagem relacional:** a separação entre a entidade `roles` e a tabela associativa com restrição de unicidade (`user_id`, `role_id`) ilustra um padrão de modelagem mais robusto do que uma simples coluna `role` no modelo `User`.

A propriedade `papel` no modelo `User` encapsula essa lógica e garante o comportamento binário esperado pela aplicação:

```

1 @property
2 def papel(self) -> str:
3     ativos = [a for a in self.role_associations if a.finished_at is
4               None]
5     for a in ativos:
6         if a.role and a.role.name.lower() == "prestador":
7             return "prestador"
8     return "cliente"

```

Listing 3: `models/user.py` – propriedade `papel`

Dessa forma, mesmo que o banco suporte múltiplos papéis ativos, a camada de aplicação resolve sempre um resultado binário, mantendo o comportamento consistente com a regra de negócio atual.

5 Detalhamento Técnico das Principais Partes do Código

5.1 Modelos SQLAlchemy – Typed Mapped Columns

Os modelos utilizam a API declarativa moderna do SQLAlchemy 2.x com `Mapped` e `mapped_column`, que traz validação de tipos em tempo de desenvolvimento e elimina

a necessidade de `Column()` explícito.

```

1 class User(UserMixin, db.Model):
2     __tablename__ = "users"
3
4     id:          Mapped[int]          = mapped_column(Integer,
5         primary_key=True)
6     name:        Mapped[str]          = mapped_column(String(100),
7         nullable=False)
8     email:       Mapped[str]          = mapped_column(String(100),
9         unique=True)
10    password:    Mapped[Optional[str]] = mapped_column(String(255))
11    cpf:         Mapped[Optional[str]] = mapped_column(String(14),
12        unique=True)
13    is_active:   Mapped[bool]          = mapped_column(Boolean,
14        default=True)
15    created_at:  Mapped[datetime]      = mapped_column(
16        DateTime(timezone=True),
17        server_default=func.now())
18
19    role_associations: Mapped[List["RoleUser"]] = relationship(
20        "RoleUser", back_populates="user", cascade="all,
21        delete-orphan")
22    addresses: Mapped[List["Address"]] = relationship(
23        "Address", back_populates="user", cascade="all,
24        delete-orphan")
25
26    def set_password(self, senha: str) -> None:
27        self.password = generate_password_hash(senha)
28
29    def check_password(self, senha: str) -> bool:
30        return bool(self.password and
31            check_password_hash(self.password, senha))
32
33    @property
34    def papel(self) -> str:
35        ativos = [a for a in self.role_associations if a.finished_at
36            is None]
37        for a in ativos:
38            if a.role and a.role.name.lower() == "prestador":
39                return "prestador"
40        return "cliente"

```

Listing 4: models/user.py – Modelo User**5.2 Blueprints e Modularização de Rotas**

Cada domínio funcional possui seu próprio Blueprint registrado na `create_app()`, garantindo separação clara de responsabilidades:

```

1 from helpnow.views.main import bp_main
2 from helpnow.views.auth import bp_auth
3 from helpnow.views.perfil import bp_perfil
4
5 def init_app(app):
6     app.register_blueprint(bp_main)
7     app.register_blueprint(bp_auth)
8     app.register_blueprint(bp_perfil)
9     app.logger.info("Blueprints registrados: main, auth, perfil")

```

Listing 5: views/__init__.py – registro de Blueprints**5.2.1 Blueprint de Autenticação**

```

1 bp_auth = Blueprint("auth", __name__, url_prefix="/auth")
2
3 @bp_auth.route("/login", methods=["GET", "POST"])
4 def login():
5     if current_user.is_authenticated:
6         return redirect(url_for("main.index"))
7     form = LoginForm()
8     if form.validate_on_submit():
9         user = User.query.filter_by(
10             email=form.email.data.strip().lower()).first()
11         if user and user.check_password(form.password.data):
12             login_user(user, remember=form.lembrar.data)
13             flash(f"Bem-vindo, {user.name.split()[0]}!", "success")
14             return redirect(request.args.get("next") or
15                             url_for("main.index"))
16         flash("E-mail ou senha incorretos.", "danger")
17     return render_template("main/login.html", form=form)

```

Listing 6: views/auth.py – Blueprint auth

5.3 Formulários e Validação com Flask-WTF

Os formulários são implementados com `FlaskForm` (que integra proteção CSRF automaticamente) e validadores customizados no padrão `validate_<campo>` do WTForms.

```

1 class CadastroForm(FlaskForm):
2     name = StringField('Nome completo',
3                         validators=[DataRequired(), Length(min=3, max=100)])
4     email = EmailField('E-mail', validators=[DataRequired(),
5                                             Email()])
6     papel = SelectField('Tipo de conta', choices=[
7         ('cliente', 'Cliente'),
8         ('prestador', 'Prestador'),
9     ], validators=[DataRequired()])
10    cpf = StringField('CPF', validators=[Length(max=14)])
11
12    def validate_cpf(self, field):
13        if self.papel.data == 'prestador':
14            if not field.data or not field.data.strip():
15                raise ValidationError('CPF obrigatorio para
16                                     prestadores.')
17            if not _RE_CPF.match(field.data.strip()):
18                raise ValidationError('Formato invalido. Use:
19                                     000.000.000-00')
20
21    def validate_email(self, field):
22        if User.query.filter_by(email=field.data).first():
23            raise ValidationError('Este e-mail ja esta cadastrado.')
```

Listing 7: forms/auth.py – CadastroForm com validação condicional

5.4 Consulta Agregada – Avaliação Média

A média das avaliações dos prestadores é calculada diretamente no banco via consulta SQL com `func.avg()`, evitando carregamento desnecessário de objetos em memória:

```

1 def _calcular_avaliacao_media(prestador_id: int):
2     resultado = (
3         db.session.query(func.avg(Solicitacao.nota))
4         .join(Servico, Solicitacao.servico_id == Servico.id)
5         .filter(
6             Servico.prestador_id == prestador_id,
7             Solicitacao.nota.isnot(None),
```

```

8         )
9         .scalar()
10    )
11    return round(float(resultado), 1) if resultado else None

```

Listing 8: views/main.py – cálculo de média com SQL agregado

5.5 Frontend – Templates Jinja2 com Bulma

Os templates utilizam herança Jinja2 com um `base.html` central que fornece estrutura, navbar e footer por meio de `partials`.

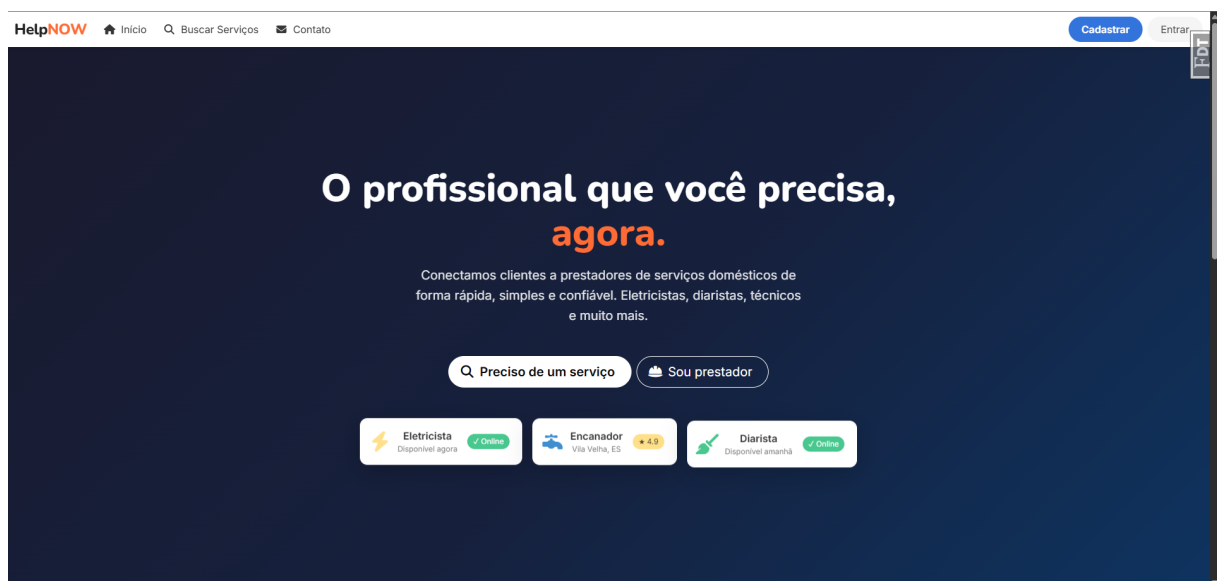


Figura 3: Página inicial – visitante não autenticado

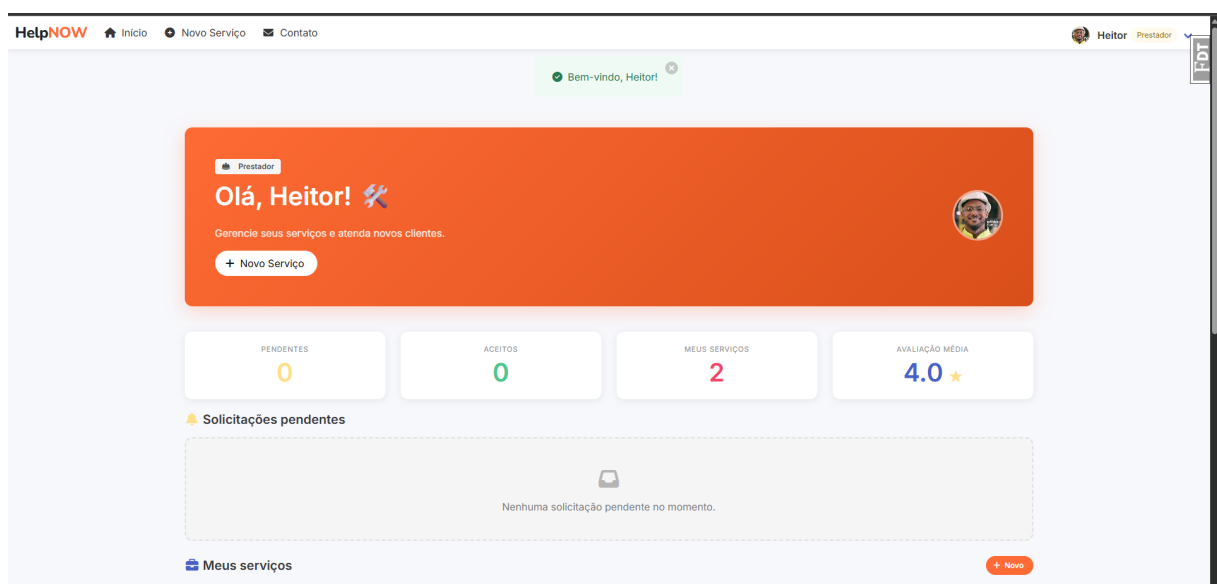


Figura 4: Dashboard do prestador

```

templates/main/
|-- base.html          % layout base (navbar, footer, flash
    messages)
|-- partials/
|   |-- navbar.html    % barra de navegacao
|   `-- footer.html    % rodape
|-- index.html         % landing page (nao autenticado)
|-- index_prestador.html % dashboard do prestador
|-- index_cliente.html  % dashboard do cliente
|-- login.html / cadastro.html
|-- buscar.html        % listagem e filtros de servicos
|-- novo_servico.html / editar_servico.html
|-- solicitar.html      % form de solicitacao de servico
|-- perfil.html         % edicao de perfil
`-- contato.html

```

Listing 9: Hierarquia de templates Jinja2

5.6 Configuração por Ambiente e Variáveis de Ambiente

O projeto usa arquivos `.env.<ambiente>` carregados pelo Invoke antes de cada operação, garantindo isolamento total entre desenvolvimento, teste e produção:

```

1 FLASK_APP=app.py
2 FLASK_ENV=development
3 FLASK_DEBUG=1
4 SECRET_KEY=dev-secret-key
5 DATABASE_URL=sqlite:///helpnow.db

```

Listing 10: `.env.dev` – variáveis de desenvolvimento

```

1 def init_app(app):
2     load_dotenv(override=True)
3     app.config['SECRET_KEY'] =
4         os.environ.get('SECRET_KEY')
5     app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get(
6         'DATABASE_URL', 'sqlite:///helpnow.db')
7     app.config['DEBUG'] = os.environ.get('FLASK_DEBUG') == '1'

```

Listing 11: `ext/config/__init__.py` – carregamento de config

6 Testes Automatizados

Os testes são implementados com `pytest` e `pytest-flask`, utilizando um banco SQLite em memória e desativando o CSRF para isolamento total de infraestrutura.

6.1 Estratégia de Testes

Tabela 3: Cobertura de testes por categoria

Categoria	Casos testados
Infraestrutura	Criação da app, carregamento de config
Rotas públicas	/, /buscar, /contato, /auth/login, /auth/cadastro (HTTP 200)
Rota inexistente	HTTP 404
Conteúdo HTML	Presença da marca “HelpNOW” na landing page
Busca c/ filtro	Categoria via query string
Formulário (POST)	Envio válido de contato → HTTP 302
Autenticação	Rota protegida redireciona para login (HTTP 302)

```

1 @pytest.fixture
2 def app():
3     application = create_app({
4         "TESTING": True,
5         "SECRET_KEY": "test-key",
6         "SQLALCHEMY_DATABASE_URI": "sqlite:///memory:",
7         "WTF_CSRF_ENABLED": False,
8     })
9     with application.app_context():
10         db.create_all()
11     return application
12
13 ROTAS_PUBLICAS = ["/", "/buscar", "/contato", "/auth/login",
14                  "/auth/cadastro"]
15
16 @pytest.mark.parametrize("rota", ROTAS_PUBLICAS)
17 def test_rotas_publicas_retornam_200(client, rota):
18     response = client.get(rota)
19     assert response.status_code == 200

```

19

```
assert b"<!DOCTYPE html>" in response.data
```

Listing 12: tests/test_main.py – fixtures e testes parametrizados

Para executar os testes:

```
invoke test
```

```

(venv) PS C:\Users\Thomás\Documents\UVA\2026\Desenvolvimento WEB\HelpNOW\helpnow-projeto-final> invoke test
[ENV] Ambiente 'TEST' carregado com sucesso a partir de: .env.test
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\Thomás\Documents\UVA\2026\Desenvolvimento WEB\HelpNOW\helpnow-projeto-final\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Thomás\Documents\UVA\2026\Desenvolvimento WEB\HelpNOW\helpnow-projeto-final
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, flask-1.3.0
collecting ... collected 16 items

tests/test_main.py::test_app_is_created PASSED [ 6%]
tests/test_main.py::test_config_is_loaded PASSED [ 12%]
tests/test_main.py::test_app_run_in_test_mode PASSED [ 18%]
tests/test_main.py::test rota_inexistente retorna 404 PASSED [ 25%]
tests/test_main.py::test rotas publicas retornam 200 [ ] PASSED [ 31%]
tests/test_main.py::test rotas publicas retornam 200 [buscar] PASSED [ 37%]
tests/test_main.py::test rotas publicas retornam 200 [contato] PASSED [ 43%]
tests/test_main.py::test rotas publicas retornam 200 [auth/login] PASSED [ 50%]
tests/test_main.py::test rotas publicas retornam 200 [auth/cadastro] PASSED [ 56%]
tests/test_main.py::test landing_page contem marca PASSED [ 62%]
tests/test_main.py::test busca sem filtro retorna pagina PASSED [ 68%]
tests/test_main.py::test busca com categoria retorna pagina PASSED [ 75%]
tests/test_main.py::test contato form valido redireciona PASSED [ 81%]
tests/test_main.py::test login get retorna formulario PASSED [ 87%]
tests/test_main.py::test cadastro get retorna formulario PASSED [ 93%]
tests/test_main.py::test rota protegida redireciona sem login PASSED [100%]

===== 16 passed in 0.82s =====
(venv) PS C:\Users\Thomás\Documents\UVA\2026\Desenvolvimento WEB\HelpNOW\helpnow-projeto-final>

```

Figura 5: Saída do pytest – todos os testes passando

7 Qualidade de Código e Boas Práticas

- **Separação de responsabilidades:** cada módulo sob `helpnow/ext/` inicializa uma única extensão Flask; `models`, `views` e `forms` estão em pacotes independentes;
- **Variáveis de ambiente:** nenhuma credencial está hard-coded no código-fonte; tudo é carregado de arquivos `.env.*` via `python-dotenv`;
- **Logging:** a aplicação usa `app.logger` nativo do Flask com nível `DEBUG` em desenvolvimento e registra eventos como o registro de blueprints;
- **Tratamento de erros:** `db.get_or_404()` é usado em todas as rotas que buscam entidades por ID, retornando HTTP 404 automaticamente; flash messages fornecem feedback contextual em cada operação;
- **Autorização:** o helper `_exige_papel()` verifica o papel do usuário antes de qualquer operação restrita, retornando redirecionamento com flash de alerta em caso de acesso indevido;
- **Hashing de senhas:** `werkzeug.security` gerencia hashing e verificação; a senha nunca é armazenada em texto plano;

- **Automação:** o arquivo `tasks.py` centraliza instalação, execução, seed de dados, testes e empacotamento em comandos Invoke reproduzíveis.

8 Como Rodar o Projeto (Via Github)

8.1 Pré-requisitos

- Python 3.9 ou superior
- pip (gerenciador de pacotes Python)
- Git (para clonar o repositório)

8.2 Passo a Passo

```
# 1. Clonar o repositório no Github
git clone https://github.com/KriggerThomas/helpnow.git
cd helpnow

# 2. Criar e ativar o ambiente virtual
py .\scripts\make_env.py
source venv/bin/activate          # Linux / macOS
venv\Scripts\activate            # Windows

# 3. Instalar dependências (modo desenvolvimento)
pip install -e ".[dev,test]"
# Alternativa com Invoke:
invoke install

# 4. Criar o banco de dados
flask create-db

# 5. (Opcional) Popular com dados de exemplo
invoke seed-dev
# Usuarios criados:
#   Cliente:      thomas@email.com   / 123456
#   Prestador:    heitor@email.com    / 123456
#   Prestador:    antony@email.com    / 123456

# 6. Iniciar o servidor de desenvolvimento
invoke run
# Acesse: http://localhost:5000
```

```
# 7. Executar os testes
invoke test
```

Listing 13: Instalação e execução do HelpNOW**Tabela 4:** Variáveis de ambiente por contexto

Arquivo	Uso	Variáveis principais
.env.dev	Desenvolvimento	FLASK_DEBUG=1, SQLite local
.env.test	Testes	TESTING=True, SQLite em memória
.env.prod	Produção	SECRET_KEY forte, DB externo

9 Principais Desafios e Como Foram Resolvidos

9.1 Validação Condicional por Papel no Formulário de Cadastro

Problema: Durante o planejamento do software, foi definido que os prestadores apenas precisam fornecer CPF e endereço na hora do cadastro, enquanto clientes não. No entanto, durante os testes funcionais do app, foi descoberto que o comportamento não seguia o esperado. Logo descobrimos que usar `Optional()` do WTForms interrompe a cadeia de validação antes dos validadores customizados, impedindo a lógica condicional.

Solução: A solução foi basicamente remover `Optional()` dos campos CPF e endereço e implementar a lógica condicional inteiramente nos métodos `validate_<campo>`. O validador executa sempre e decide o que exigir com base no valor de `self.papel.data`.

9.2 Número de Endereço como String

Problema: Depois de algumas pesquisas, percebemos que alguns endereços brasileiros frequentemente usam formatos não numéricos como “S/N”, “123-A” ou “Ap. 5”. Um campo `Integer` no banco rejeitaria esses valores.

Solução: Simplesmente o campo `number` da tabela `address` foi modelado como `String(15)`, preservando todos os formatos sem perda de dados.

9.3 Desativação do CSRF nos Testes

Problema: Nas primeiras tentativas de rodar os testes, notamos que o CSRF do Flask-WTF bloqueava requisições POST nos testes automatizados, exigindo geração de token a cada chamada.

Solução: Foi preciso apenas ajustar a função `create_app()` para verificar o flag `TESTING` e omitir a inicialização do Flask-WTF nesse contexto. O dicionário de configuração injetado no fixture também define `WTF_CSRF_ENABLED: False` como camada adicional de segurança.

10 Conclusão

Poder praticar desenvolvendo o HelpNOW nos ajudou a aplicar, de forma integrada, os principais conceitos da disciplina durante todo o semestre e as melhores tecnologias modernas para desenvolvimento Web, como arquitetura modular com Application Factory e Blueprints, persistência com SQLAlchemy 2.x, formulários com validação robusta via Flask-WTF, templates responsivos com Jinja2 e Bulma, e cobertura de testes com pytest.

Vale ressaltar que o trabalho está organizado de tal forma que a substituição do banco SQLite por PostgreSQL em produção requer apenas a alteração da variável `DATABASE_URL`, sem nenhuma modificação no código-fonte, demonstrando a aderência ao princípio de configuração por ambiente.

Como estudante de Ciência da Computação, foi um prazer poder fazer parte desse projeto, mesmo limitando se apenas a um projeto acadêmico como avaliação integral do segundo bimestre da disciplina de desenvolvimento de sistemas para Web.

Atenciosamente,

Thomás.

Vila Velha, junho de 2026.

Antony Novais · Heitor Rodrigues · Thomás Kriger